Scenario based Programs on passing object reference to the method and Factory Method :

A BLC class called Customer is given to you.

The task is to find the applicable Credit card Type and create CardType object based on the Credit Points of a customer.

Define the following for the class.

Attributes :
customerName : String,private
creditPoints: int, private

Constructor :
parameterizedConstructor: for both customerName & creditPoints in that order.

Methods :
Name of the method : getCreditPoints
Return Type : int
Modifier : public
Task : This method must return creditPoints

Name of the method : toString, Override it.
Return Type : String
Task : return only customerName from this.

Create another class called CardType. Define the following for the class

Attributes :
customer : Customer, private
cardType : String, private
Constructor :
parameterizedConstructor: for customer and cardType attributes in that order

Methods :
Name of the method : toString. Override this.
Return type : String
Modifier : public
Task : Return the string in the following format
"The Customer <this.customer> is eligible for <this.cardType>" card.

Create another class by name CardAllotter and define the following for the class.

Method :
Name Of the method : getOfferedCard
Return type : CardType [Factory Method]
Access Modifiers :public
Parameter : Customer object

Task : Create and return a CardType object after logically feeding cardType from creditPoints as per the below rules.
creditPoints cardType
0 - 500 Silver
501 - 1000 Gold
1001 - Platinum
150 - Elite

Create an BLC class which contains Main method to test the working of the above.

```
package com.ravi.scenario_based;

public class Customer
{
    private String customerName;
    private int creditPoints;

    public Customer(String customerName, int creditPoints)
    {
        super();
        this.customerName = customerName;
        this.creditPoints = creditPoints;
    }

    public int getCreditPoints()
    {
        return this.creditPoints;
    }

    @Override
    public String toString()
    {
        return this.customerName;
    }
}

package com.ravi.scenario_based;

public class CardType
{
    private Customer customer; //HAS-A
    private String cardType; //HAS-A
                                //Initializer
    public CardType(Customer customer, String cardType)
    {
        super();
        this.customer = customer;
        this.cardType = cardType;
    }

    @Override
    public String toString()
    {
        return "The Customer "+this.customer+" is eligible for "+this.cardType+" card.";
    }
}

package com.ravi.scenario_based;

public class CardAllotter
{
    public static CardType getOfferedCard(Customer obj) //obj = new
    {
        int creditPoints = obj.getCreditPoints();
        if(creditPoints >=0 && creditPoints <=500)
        {
            return new CardType(obj, "Silver");
        }
        else if(creditPoints >500 && creditPoints <=1000)
        {
            return new CardType(obj, "Gold");
        }
        else if(creditPoints > 1000)
        {
            return new CardType(obj, "Platinum");
        }
        else
        {
            return new CardType(obj, "Elite");
        }
    }
}

package com.ravi.scenario_based;

import java.util.Scanner;

public class CreditCardApp {

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter Customer Name :");
        String customerName = sc.next();

        System.out.println("Enter customer credit point :");
        int creditPoint = Integer.parseInt(sc.next());

        Customer c = new Customer(name, creditPoint); //1000
        CardType cType=CardAllotter.getOfferedCard(c);

        System.out.println(cType);

        sc.close();

    }

}
```

Constructor Overloding :

* If we write two or more constructor in the **same class with different parameter list then it is called Constructor overloading.**

* The parameter list must be differ in :
a) Number of Parameters
b) The data type of the parameter
c) Order of the Parameter [Sequences of the parameter]

Example:

```
package com.ravi.constructor_overloading;

public class Demo
{

    public Demo(float x)
    {

    }

    public Demo(double x)
    {

    }

    public Demo(int x)
    {

    }

    public Demo(int x, int y)
    {

    }

    public Demo(int x, long y)
    {

    }

    public Demo(long x, int y)
    {

    }
}
```

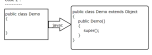
* THE FIRST LINE OF ANY CONSTRUCTOR (ENNANCEMENT FROM JDK 25+) IS MEANT FOR CALLING ANOTHER CONSTRUCTOR EITHER OF SAME CLASS (this()) OR SUPER CLASS (super())

* the first line of any constructor is reserved for either super() OR this(). If a developer does not write either super() OR this() to the first line of constructor then automatically java compiler will add super() to the first line of constructor. (javac will never add this())

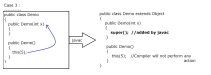
Case 1 :

Demo.java

Case 2 :



Case 3 :



* The main purpose of this() (this ref) to call the current class appropriate constructor. Java compiler will never add this().

* this() (calling the current class constructor) must be first line as well as must be inside constructor body, we cannot use this() inside the nonlocal body.

* Calling from one constructor to another constructor is called constructor chaining.

```
package com.ravi.constructor_overloading;

class Addition
{
    public Addition(int x, int y)
    {
        super();
        System.out.println("Sum of two integer is :"+(x+y));
    }

    public Addition(String x, String y)
    {
        this(12,20);
        System.out.println("Concatenation of two String is :"+(x+y));
    }

    public Addition(double x, double y)
    {
        this("Ravi", "31");
        System.out.println("Sum of two double is :"+(x+y));
    }
}

public class ConstructorOverloading
{
    public static void main(String[] args)
    {
        Addition ob = new Addition(2.3, 6.7);

    }
}
```